

L03b Image Classification with SVM

The purpose of this lab was to practice a basic image classification workflow using the CIFAR-10 dataset and a Support Vector Machine (SVM). CIFAR-10 contains small 32 by 32 color images from ten object classes, including animals and vehicles (Krizhevsky, 2009; TensorFlow Developers, n.d.). For this notebook, I worked with a smaller subset of three classes: cat, dog, and ship.

Using a subset made the lab more practical in Google Colab because SVM training can be slow when the dataset becomes large. The selected classes also created a useful comparison. Cats and dogs are visually similar, so I expected the model to confuse those classes more often. Ships are visually more distinct, so I expected that class to be easier for the model.



Figure 1. Original CIFAR-10 sample images from the selected classes used in the lab

Reflection on Learning

My understanding of an SVM is that it is a supervised machine learning algorithm that learns decision boundaries between labeled groups of data. In classification, the SVM attempts to find a boundary, or hyperplane, that separates one class from another. The scikit-learn documentation describes support vector machines as supervised methods used for classification, regression, and outlier detection, and notes that they can be effective in high-dimensional spaces (scikit-learn Developers, n.d.-a).

This concept mattered in the lab because images become high-dimensional after preprocessing. A 32 by 32 grayscale image contains 1,024 pixel values after flattening. Each pixel value acts like a separate feature, so the SVM is not looking at a picture the way a person does. It is comparing long numerical vectors and learning which patterns of values are associated with cats, dogs, or ships.

Data Preparation Process

The data preparation steps were important because the SVM could only learn from the numerical representation created before training. The main preparation steps were:

- Loaded the CIFAR-10 dataset using TensorFlow/Keras.
- Selected only cat, dog, and ship images to reduce runtime and keep the results easier to interpret.

- Visualized sample images to confirm that the dataset loaded correctly.
- Converted RGB images to grayscale so each pixel had one intensity value instead of three color-channel values.
- Normalized pixel values from 0-255 to 0-1 to place the features on a more consistent numerical scale.
- Flattened each 32 by 32 grayscale image into a 1,024-value feature vector for scikit-learn.

The grayscale step simplified the data, but it also removed color information. That tradeoff made the lab easier to run, but it may have limited the model's ability to distinguish classes that depend partly on color or texture. Flattening was also necessary for the classical machine learning workflow, but it caused the image to lose some of its natural spatial structure.

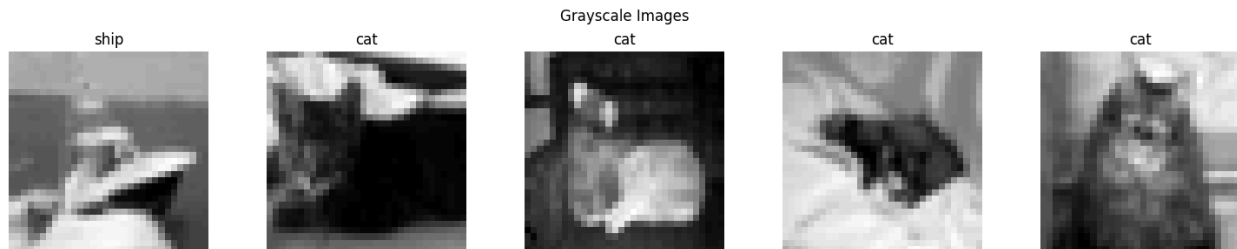


Figure 2. Grayscale versions of the selected CIFAR-10 images after preprocessing.

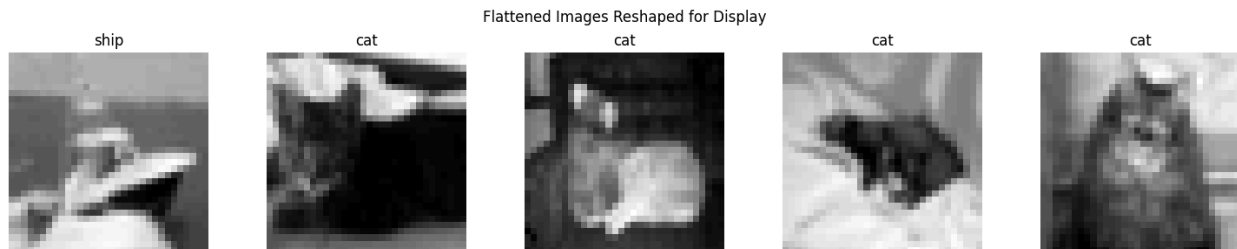


Figure 3. Flattened images reshaped for display. The SVM receives these as one-dimensional feature vectors, even though they are shown here as images for interpretation.

Model Training and Evaluation

The notebook trained an SVM classifier using scikit-learn's SVC model with a linear kernel. A linear kernel was appropriate for a beginner lab because it keeps the model easier to explain and avoids adding more complexity before the basic workflow is understood. The scikit-learn documentation notes that SVC is based on libsvm and that fit time can scale poorly as the number of samples increases, which explains why image classification with SVMs can become slow (scikit-learn Developers, n.d.-b).

After training, the model predicted labels for the held-out test set. The final accuracy was 0.547, or 54.7%. Since the task used three balanced classes, random guessing would be expected to perform around 33.3%. The model therefore learned useful visual patterns, but its performance was still limited.

The confusion matrix gave more detail than accuracy alone. Out of 1,000 ship images, 684 were correctly predicted as ships. In comparison, 479 cat images and 478 dog images were correctly classified. The largest errors occurred between cats and dogs: 347 cats were predicted as dogs, and 344 dogs were predicted as cats. This supports the expectation that the model would struggle more with visually similar animal classes than with ships.

The prediction examples also made the results easier to understand. Some images were classified correctly, while others were confused even when the object was recognizable to a human viewer. This showed that the model's numeric representation was not equivalent to human visual understanding.

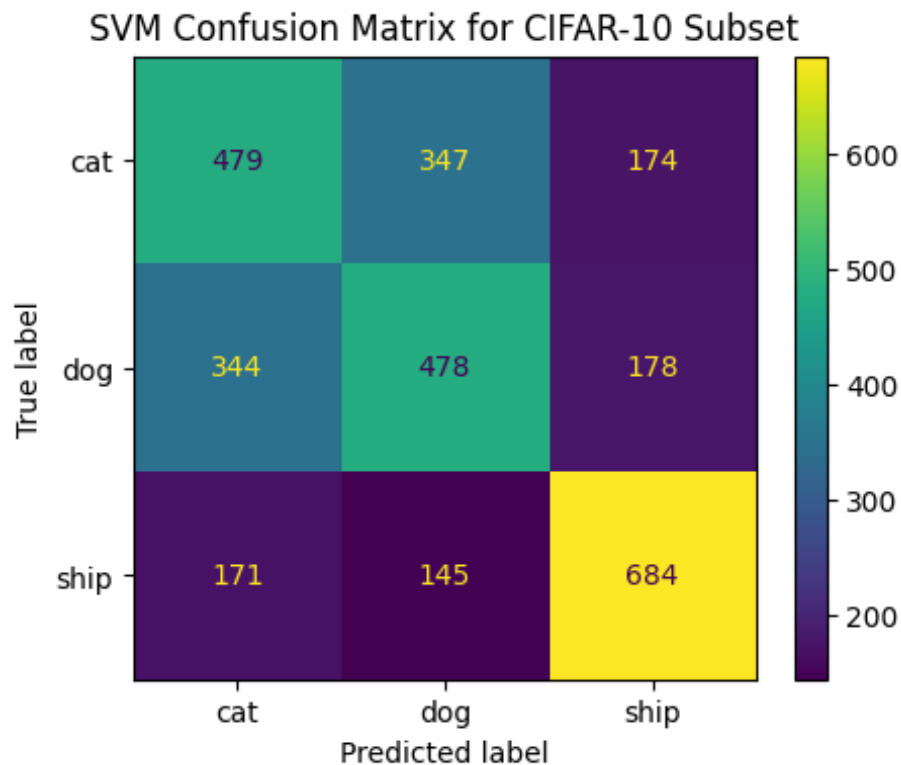


Figure 4. Confusion matrix for the SVM classifier on the selected CIFAR-10 classes.

Challenges and Insights

The main challenge in this lab was runtime. Downloading CIFAR-10 in Colab took a long time, and SVM training can also be computationally expensive when many images are used. Using a smaller class subset made the lab feasible while preserving the main learning objective. The lab also helped me understand that the model does not see a cat, dog, or ship as a full object the way a person does. After grayscale conversion and flattening, the SVM only sees

Kate Leemann
ITAI 1378
Professor McManus
June 17, 2026

numerical pixel values, so preprocessing directly controls what information the model can use. This connects with how I think about data in genetics and R&D work, where raw data must be cleaned, transformed, and checked before analysis. In both cases, the preparation step strongly affects the quality of the final result.

The final accuracy of 54.7% showed that the SVM learned meaningful patterns, especially for ships, but struggled with visually similar classes such as cats and dogs. Overall, this lab showed the complete classical image classification workflow: loading data, preprocessing images, training a model, making predictions, and evaluating errors. It also showed the limits of flattened grayscale pixels for image recognition and why more advanced methods, such as convolutional neural networks, are usually better suited for complex computer vision tasks.

References

Krizhevsky, A. (2009). *Learning multiple layers of features from tiny images*. University of Toronto.

<https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>

Krizhevsky, A., Nair, V., & Hinton, G. *CIFAR-10 and CIFAR-100 datasets*.

<https://www.cs.toronto.edu/~kriz/cifar.html>

scikit-learn developers. (2026a). *1.4. Support Vector Machines*.

<https://scikit-learn.org/stable/modules/svm.html>

scikit-learn developers. (2026b). *sklearn.svm.SVC*.

<https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>

TensorFlow Developers. (2024). *tf.keras.datasets.cifar10.load_data*.

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/cifar10/load_data